Predicting Stock Prices Using RNN with LSTM

TA1 Assignment | Tesla (TSLA) Stock | 2019–2024

Imports

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import mplfinance as mpf

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping,
ReduceLROnPlateau

import warnings
warnings.filterwarnings('ignore')

# Install mplfinance if needed
# !pip install mplfinance

print('All libraries imported successfully!')
```

Task 1: Data Preparation

Steps: Load dataset → Filter 2019-2024 → Feature Selection → Normalization & Scaling

```
# 1.1 Load Dataset
df = pd.read_csv('Tesla Stock Dataset.csv')
df['Date'] = pd.to_datetime(df['Date'])

# Filter: 2019 to 2024 (5 years of historical data as required)
df = df[(df['Date'] >= '2019-01-01') & (df['Date'] <= '2024-12-31')]
df = df.sort_values('Date').reset_index(drop=True)

print('Dataset shape:', df.shape)
print('Date range:', df['Date'].min(), 'to', df['Date'].max())
print('\nFirst 5 rows:')
```

```python
print(df.head())
print('\nMissing values:\n', df.isnull().sum())

# 1.2 Feature Selection
# Using multiple features: Open, High, Low, Close, Volume
# Close price is our prediction target
features = ['Open', 'High', 'Low', 'Close', 'Volume']
df_features = df[features].copy()
df_features.dropna(inplace=True)

print('Selected features:', features)
print('Shape after feature selection:', df_features.shape)
print('\nStatistical Summary:')
print(df_features.describe())

# 1.3 Normalization & Scaling
# MinMaxScaler scales all values to [0, 1]
# Separate scaler for Close price for inverse transform during
prediction

scaler_all   = MinMaxScaler(feature_range=(0, 1))
scaler_close = MinMaxScaler(feature_range=(0, 1))

scaled_all   = scaler_all.fit_transform(df_features)
scaled_close = scaler_close.fit_transform(df_features[['Close']])

print('Scaling complete!')
print(f'Original Close range : {df_features["Close"].min():.2f} to
{df_features["Close"].max():.2f}')
print(f'Scaled  Close range  : {scaled_all[:, 3].min():.4f} to
{scaled_all[:, 3].max():.4f}')

# 1.4 Create Sequences for LSTM
# 60 trading days lookback window
TIME_STEP = 60

def create_sequences(data, target_col_idx, time_step=60):
    X, y = [], []
    for i in range(len(data) - time_step - 1):
        X.append(data[i : i + time_step, :])           # all features
        y.append(data[i + time_step, target_col_idx])  # predict Close
    return np.array(X), np.array(y)

# Train / Test Split: 80% / 20%
train_size = int(len(scaled_all) * 0.8)
train_data = scaled_all[:train_size]
test_data  = scaled_all[train_size:]

CLOSE_IDX = features.index('Close')  # index = 3

X_train, y_train = create_sequences(train_data, CLOSE_IDX, TIME_STEP)
```

```
X_test,  y_test  = create_sequences(test_data,  CLOSE_IDX, TIME_STEP)

print(f'Training samples : {X_train.shape}')  # (samples, 60, 5)
print(f'Test samples     : {X_test.shape}')
print(f'Train rows: {train_size} | Test rows: {len(scaled_all) -
train_size}')
```

Task 2: Model Development

RNN with LSTM cells — addresses vanishing gradient problem and retains long-term memory

```
# 2.1 Build LSTM Model
n_features = X_train.shape[2]  # 5 features

model = Sequential([
    # Layer 1: LSTM — return_sequences=True feeds output to next LSTM
    LSTM(128, return_sequences=True, input_shape=(TIME_STEP,
n_features)),
    Dropout(0.2),

    # Layer 2: LSTM
    LSTM(64, return_sequences=True),
    Dropout(0.2),

    # Layer 3: LSTM — final LSTM layer
    LSTM(32, return_sequences=False),
    Dropout(0.2),

    # Output layer: single neuron for price prediction
    Dense(1)
])

model.compile(optimizer='adam', loss='mean_squared_error')
model.summary()
```

Task 3: Training

Optimizer: Adam | Loss: Mean Squared Error | Callbacks: EarlyStopping + ReduceLROnPlateau

```
# 3.1 Train the Model
early_stop = EarlyStopping(monitor='val_loss', patience=5,
restore_best_weights=True)
reduce_lr  = ReduceLROnPlateau(monitor='val_loss', factor=0.5,
patience=3, min_lr=1e-6)
```

```
history = model.fit(
    X_train, y_train,
    epochs           = 50,
    batch_size       = 32,
    validation_split = 0.1,
    callbacks        = [early_stop, reduce_lr],
    verbose          = 1
)

print('Training complete!')
print(f'Best val_loss: {min(history.history["val_loss"]):.6f}')

# 3.2 Plot Training & Validation Loss
plt.figure(figsize=(10, 4))
plt.plot(history.history['loss'],     label='Train Loss',
color='steelblue',  linewidth=2)
plt.plot(history.history['val_loss'], label='Val Loss',
color='darkorange', linewidth=2)
plt.title('Model Training Loss (MSE) over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Loss (MSE)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Task 4: Prediction & Evaluation

```
# 4.1 Generate Predictions
train_pred_scaled = model.predict(X_train)
test_pred_scaled  = model.predict(X_test)

# Inverse transform back to original price scale
train_predict = scaler_close.inverse_transform(train_pred_scaled)
test_predict  = scaler_close.inverse_transform(test_pred_scaled)

y_train_actual = scaler_close.inverse_transform(y_train.reshape(-1,
1))
y_test_actual  = scaler_close.inverse_transform(y_test.reshape(-1, 1))

print('Predictions generated!')
print(f'Test prediction range: ${test_predict.min():.2f} to $
{test_predict.max():.2f}')

# 4.2 Evaluation Metrics: MAE, RMSE, MAPE
mae  = mean_absolute_error(y_test_actual, test_predict)
rmse = np.sqrt(mean_squared_error(y_test_actual, test_predict))
mape = np.mean(np.abs((y_test_actual - test_predict) / y_test_actual))
```

```
* 100

print('=' * 45)
print('        MODEL EVALUATION METRICS')
print('=' * 45)
print(f'  MAE  (Mean Absolute Error)       : ${mae:.4f}')
print(f'  RMSE (Root Mean Squared Error)   : ${rmse:.4f}')
print(f'  MAPE (Mean Abs Percentage Error) : {mape:.4f}%')
print('=' * 45)
print(f'  Accuracy (100 - MAPE)            : {100 - mape:.2f}%')

# 4.3 Line Chart: Actual vs Predicted (Test Set)
plt.figure(figsize=(12, 5))
plt.plot(y_test_actual, label='Actual Price',    color='steelblue',
linewidth=2)
plt.plot(test_predict,  label='Predicted Price', color='darkorange',
linewidth=2, linestyle='--')
plt.fill_between(range(len(y_test_actual)),
                 y_test_actual.flatten(), test_predict.flatten(),
                 alpha=0.1, color='gray')
plt.title('Tesla (TSLA) Stock Price Prediction using LSTM')
plt.xlabel('Time Steps (Trading Days)')
plt.ylabel('Stock Price (USD)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# 4.4 Full Timeline: Train + Test Predictions
actual_prices = scaler_close.inverse_transform(scaled_close)

train_plot = np.empty_like(scaled_close)
train_plot[:] = np.nan
train_plot[TIME_STEP : len(train_predict) + TIME_STEP, 0] =
train_predict.flatten()

test_plot = np.empty_like(scaled_close)
test_plot[:] = np.nan
offset = len(train_predict) + (TIME_STEP * 2) + 1
test_plot[offset : offset + len(test_predict), 0] =
test_predict.flatten()

plt.figure(figsize=(14, 6))
plt.plot(actual_prices, label='Actual Price',     color='steelblue',
linewidth=1.5)
plt.plot(train_plot,    label='Train Prediction', color='limegreen',
linewidth=1.5)
plt.plot(test_plot,     label='Test Prediction',  color='darkorange',
linewidth=1.5)
plt.title('Tesla Stock: Full Price History with Train & Test
```

```
Predictions')
plt.xlabel('Trading Days (2019-2024)')
plt.ylabel('Stock Price (USD)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# 4.5 Candlestick Chart (last 60 trading days)
# Uses mplfinance library: pip install mplfinance

df_candle = df[['Date', 'Open', 'High', 'Low', 'Close',
'Volume']].copy()
df_candle = df_candle.set_index('Date').tail(60)

mpf.plot(df_candle,
         type='candle',
         style='charles',
         title='Tesla — Candlestick Chart (Last 60 Trading Days)',
         ylabel='Price (USD)',
         volume=True,
         figsize=(14, 7))

# 4.6 Future Stock Price Forecast (next 30 days)
# Starts from the last known 60-day window and predicts forward

FUTURE_DAYS   = 30
n_features    = scaled_all.shape[1]
last_sequence = scaled_all[-TIME_STEP:].copy()
future_prices = []

for _ in range(FUTURE_DAYS):
    seq_input = last_sequence.reshape(1, TIME_STEP, n_features)
    next_pred = model.predict(seq_input, verbose=0)[0, 0]
    future_prices.append(next_pred)
    # Slide window forward
    new_row = last_sequence[-1].copy()
    new_row[CLOSE_IDX] = next_pred
    last_sequence = np.vstack([last_sequence[1:], new_row])

future_prices_actual = scaler_close.inverse_transform(
    np.array(future_prices).reshape(-1, 1)
)

last_price = df['Close'].iloc[-1]
print(f'Last known price : ${last_price:.2f}')
print(f'Forecast range   : ${future_prices_actual.min():.2f} to $
{future_prices_actual.max():.2f}')

# Plot forecast
```

```
past_60 = y_test_actual[-60:].flatten()
plt.figure(figsize=(10, 5))
plt.plot(range(60), past_60, label='Recent Actual', color='steelblue',
linewidth=2)
plt.plot(range(60, 60 + FUTURE_DAYS), future_prices_actual.flatten(),
         label='30-Day Forecast', color='tomato', linewidth=2,
linestyle='--')
plt.axvline(x=60, color='gray', linestyle=':', label='Forecast Start')
plt.title('Tesla Stock — 30-Day Future Price Forecast')
plt.xlabel('Days')
plt.ylabel('Price (USD)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Summary & Observations

Metric	Description
MAE	Average dollar error per prediction
RMSE	Penalizes large errors more than MAE
MAPE	Percentage error — model accuracy

Key Observations:

1. **LSTM effectiveness**: The 3-layer LSTM captures both short and long-term price patterns, solving the vanishing gradient problem in standard RNNs.
2. **Feature selection**: Using Open, High, Low, Close, and Volume provides richer market context vs Close alone.
3. **Normalization**: MinMaxScaler ensures features contribute equally during training.
4. **EarlyStopping + ReduceLR**: Prevents overfitting and helps converge to a better minimum.
5. **30-day Forecast**: Demonstrates the model generalizing beyond the training window.